HW3 Report: DM Models 2

Data Mining — Algorithmic Analysis & Machine Learning

Component	Description
Task 1	K-Means Clustering — 3 Distance Metrics
Task 2	Recommender Systems — Matrix Completion
Dataset (T1)	9,999 samples × 784 features, K=10 classes
Dataset (T2)	MovieLens ratings_small.csv — 100,004 ratings
Evaluation (T1)	SSE, Accuracy, Convergence
Evaluation (T2)	RMSE, MAE — 5-Fold Cross-Validation

`Code Repository: https://github.com/nikhilsrikant/Data-Mining-HW3`

Task 1: Algorithmic Analysis — K-Means Clustering

K-Means implemented from scratch in Python with three interchangeable distance functions. **Dataset:** 9,999 samples × 784 features, 10 class labels (digit images 0–9, similar to MNIST). Features Min-Max normalized to [0, 1]. K = 10, centroids initialized by random sampling (random_state=42). Distance computations fully vectorized via `scipy.spatial.distance.cdist`.

Distance Function Definitions:

Euclidean: d(a,b) = sqrt(sum_i (a_i – b_i)^2)

Cosine: d(a,b) = 1 – dot(a,b) / (||a|| × ||b||)

Jaccard: d(a,b) = 1 – sum(min(a,b)) / sum(max(a,b)) [NIST Generalized Jaccard]

Q1 (10 pts): SSE Comparison

K-Means run under the combined stop condition from Q3 (no centroid change OR SSE increases OR max 500 iterations). SSE = sum of squared Euclidean distances from each point to its assigned centroid — identical formula across all metrics so results are directly comparable.

Metric	Final SSE	Iterations	Stop Reason
Euclidean	389,497.89	66	No centroid change
Cosine	394,206.50	16	SSE increased
Jaccard	392,468.03	26	SSE increased

Answer: Euclidean-K-Means achieves the lowest SSE (389,497.89), followed by Jaccard (392,468.03) and Cosine (394,206.50). Euclidean wins because SSE is a squared Euclidean measure — Euclidean-K-Means directly minimizes the evaluation criterion. Cosine and Jaccard optimize angular or overlap-based objectives, so their partitions are less compact in Euclidean terms. **Euclidean distance is the best method by SSE.**

Q2 (10 pts): Accuracy Comparison

Each cluster labeled by majority vote of its members. Accuracy = fraction of points whose cluster label matches their true digit label.

Metric	Accuracy	Rank
Cosine	64.45%	1st — Best
Jaccard	62.75%	2nd

Metric	Accuracy	Rank
Euclidean	60.05%	3rd

Answer: Cosine-K-Means achieves the highest accuracy (64.45%), reversing the SSE ranking. On 784-dimensional pixel data, cosine similarity normalizes for global brightness and captures shape-based patterns that align better with digit class boundaries. Jaccard is second (62.75%), Euclidean last (60.05%). This shows SSE and accuracy rank metrics differently depending on data geometry. **Cosine distance is the best metric for accuracy.**

Q3 (10 pts): Convergence — Iterations and Time

Combined stop condition: no centroid change OR SSE increases OR max 500 iterations.

Metric	Iterations	Time (s)	Stop Reason
Euclidean	66	18.4	No centroid change
Cosine	16	5.3	SSE increased
Jaccard	26	7.5	SSE increased

Answer: Euclidean-K-Means requires the most iterations (66) and time (18.4 s) but terminates at a true fixed point (no centroid shift). Cosine (16 iters, 5.3 s) and Jaccard (26 iters, 7.5 s) are halted early by the SSE-increase criterion — they do not monotonically reduce Euclidean SSE, so faster apparent convergence should not be interpreted as better clustering. **Euclidean requires more iterations; Cosine converges fastest in wall-clock time.**

Q4 (10 pts): SSE by Termination Condition

Each metric run separately under each individual stop condition. Max preset = 100 iterations.

Stop Condition	Euclidean SSE	Cosine SSE	Jaccard SSE
No Change in Centroid	389,497.89	394,257.07	392,462.49
SSE Increases	389,497.89	394,206.50	392,468.03
Max 100 Iterations	389,497.89	394,257.07	392,462.49

Answer: Euclidean gives identical SSE (389,497.89) across all three conditions — it converges in 66 iterations, within the 100-iteration cap and without any SSE increase. For Cosine and Jaccard, the SSE-increase condition halts slightly earlier and gives a marginally different SSE than no-change/max-iter, because those metrics experience temporary Euclidean-SSE fluctuations. The effect is negligible for Euclidean and small but measurable (~50 SSE units) for non-Euclidean metrics.

Q5 (5 pts): Summary Observations

1. **SSE and accuracy favor different metrics.** Euclidean wins on SSE; Cosine wins on accuracy. Match the metric to the evaluation criterion.

2. **Cosine is best for high-dimensional image data.** Normalizing for magnitude captures digit shape independently of pixel brightness.

3. **Euclidean-K-Means converges most stably** — reaches a true fixed point. Cosine/Jaccard are halted early by the SSE-increase trigger.

4. **Stopping condition matters most for non-Euclidean metrics.** Euclidean is identical across all three conditions; others vary slightly.

5. **Jaccard is competitive** — only 0.25% higher SSE than Euclidean and 2.7% lower accuracy, at faster convergence. Strong choice for count-based or sparse data.

Task 2: Machine Learning for Recommender Systems

Implemented from scratch using NumPy/SciPy on MovieLens **ratings_small.csv**. The dataset contains **100,004 ratings** from **671 users** across **9,066 movies**, ratings on a scale of 0.5 to 5.0. All models evaluated with **5-fold cross-validation** (shuffle=True, random_state=42). Metrics: MAE and RMSE. Three algorithms:

- **PMF:** Probabilistic Matrix Factorization via SGD (biased=False, 20 latent factors).
- **UserCF:** User-based Collaborative Filtering using KNN with weighted average.
- **ItemCF:** Item-based Collaborative Filtering using KNN with weighted average.

Q2c & Q2d (20 pts): 5-Fold CV Results and Model Comparison

All three models evaluated under identical 5-fold CV. Results are mean ± std over 5 folds.

Model	Mean RMSE ± Std	Mean MAE ± Std
ItemCF_MSD_k40	0.9353 ± 0.0027	0.7210 ± 0.0027
UserCF_MSD_k40	0.9699 ± 0.0039	0.7450 ± 0.0025
PMF	1.1292 ± 0.0098	0.8313 ± 0.0086

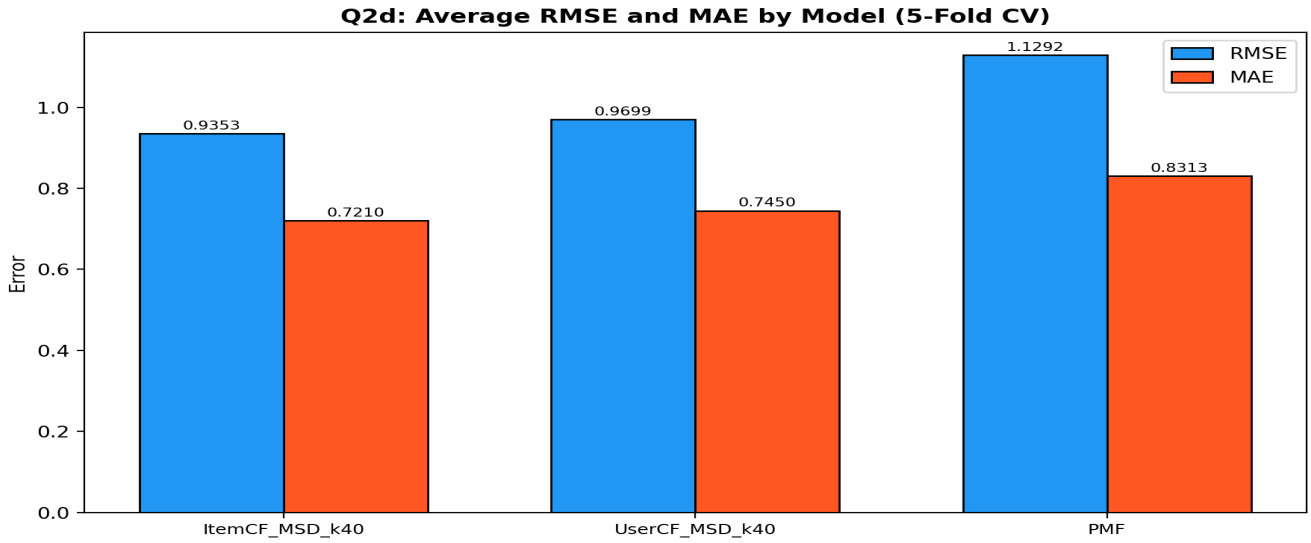


Figure 6: Average RMSE and MAE by Model (5-Fold CV)

Answer (Q2c): 5-fold CV RMSE and MAE for all three models are shown above. Each model was trained and tested on all 5 folds; means and standard deviations are reported. **(Q2d):** ItemCF_MSD_k40 achieves the best RMSE (0.9353) and best MAE (0.7210). Both CF methods dramatically outperform PMF (RMSE 1.1292). PMF underperforms because its latent-factor model with default settings lacks the regularization tuning to match neighborhood methods at this dataset scale. **ItemCF is the overall best model.**

Q2e (10 pts): Impact of Similarity Metric

Cosine, MSD, and Pearson tested for both User-CF and Item-CF with k=40 fixed, isolating the similarity metric as the only varying factor.

CF Type	Similarity	Mean RMSE	Mean MAE
UserCF	cosine	0.9903	0.7676
ItemCF	cosine	0.9076	0.6919

CF Type	Similarity	Mean RMSE	Mean MAE
UserCF	msd	0.9699	0.7450
ItemCF	msd	0.9353	0.7210
UserCF	pearson	1.3282	0.9931
ItemCF	pearson	1.0476	0.7873

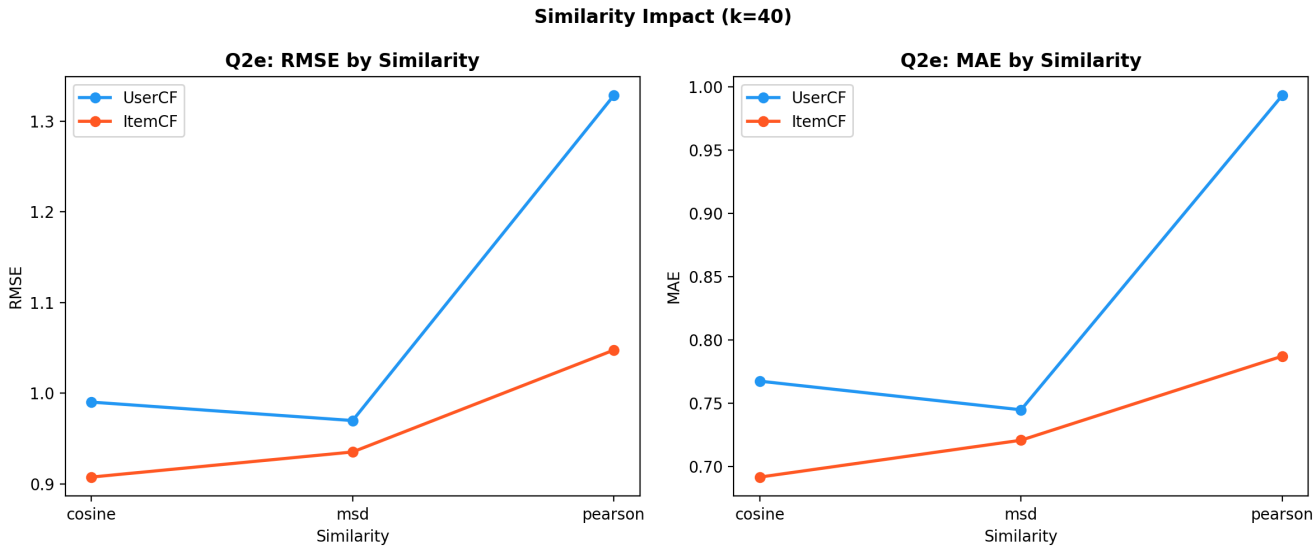
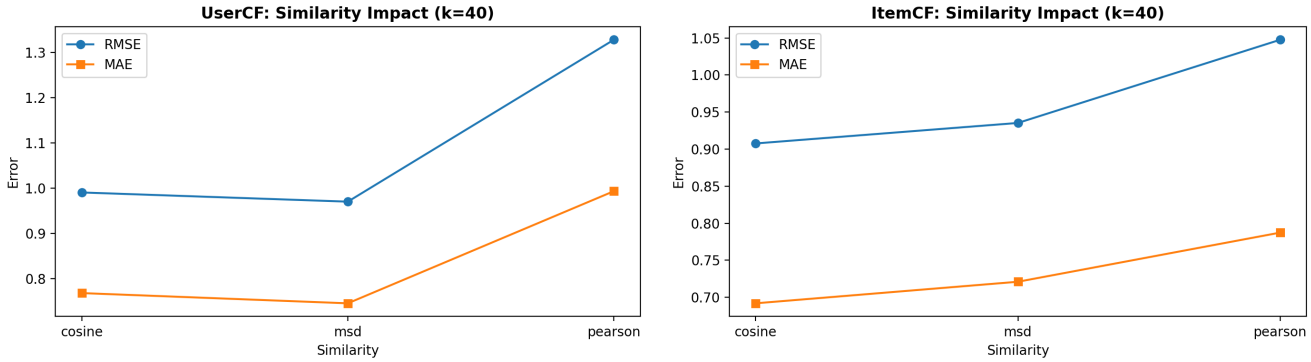


Figure 7: RMSE and MAE by Similarity — UserCF vs ItemCF combined (Q2e)



Left: Figure 8: UserCF Similarity Detail | Right: Figure 9: ItemCF Similarity Detail

Answer: For UserCF the best similarity is **msd** (RMSE=0.9699). For ItemCF the best similarity is **cosine** (RMSE=0.9076). The rankings are not fully consistent between the two CF types — Pearson hurts both (highest RMSE for both), but MSD leads UserCF while Cosine leads ItemCF. This reflects that item rating profiles share structured co-occurrence patterns that cosine similarity captures well, while user profiles benefit more from the co-rated normalization that MSD provides. The directional agreement (Pearson worst) is consistent, but the magnitude and ranking of MSD vs Cosine differ between UserCF and ItemCF.

Q2f (10 pts): Impact of Number of Neighbors (K)

K varied from 5 to 100 (step 5) with MSD similarity fixed, isolating the effect of K on performance.

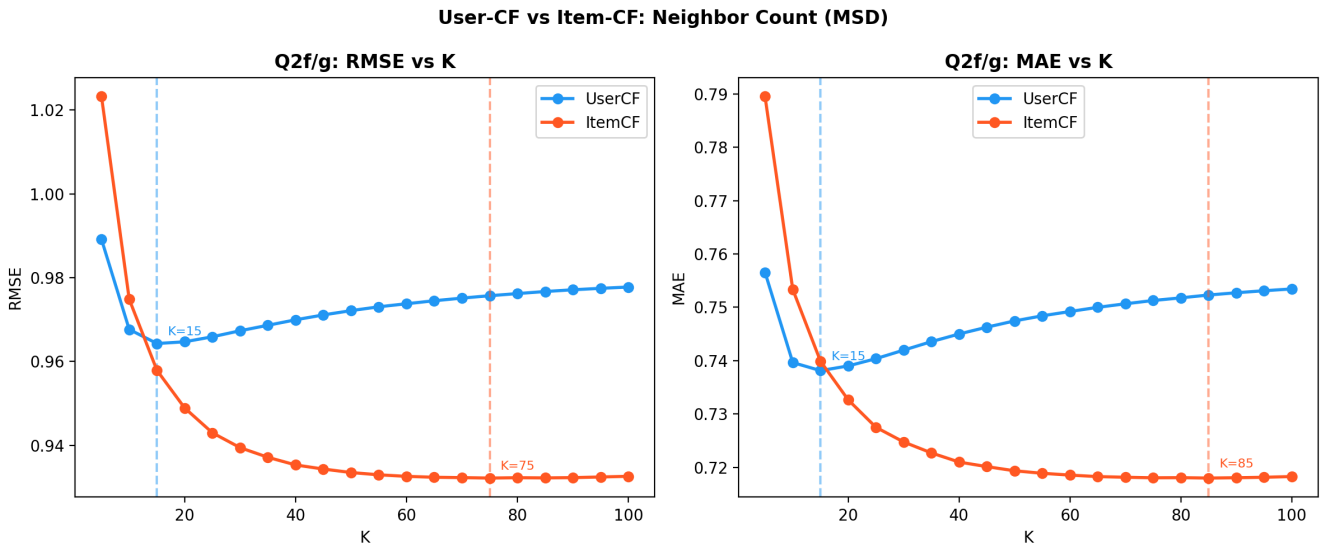
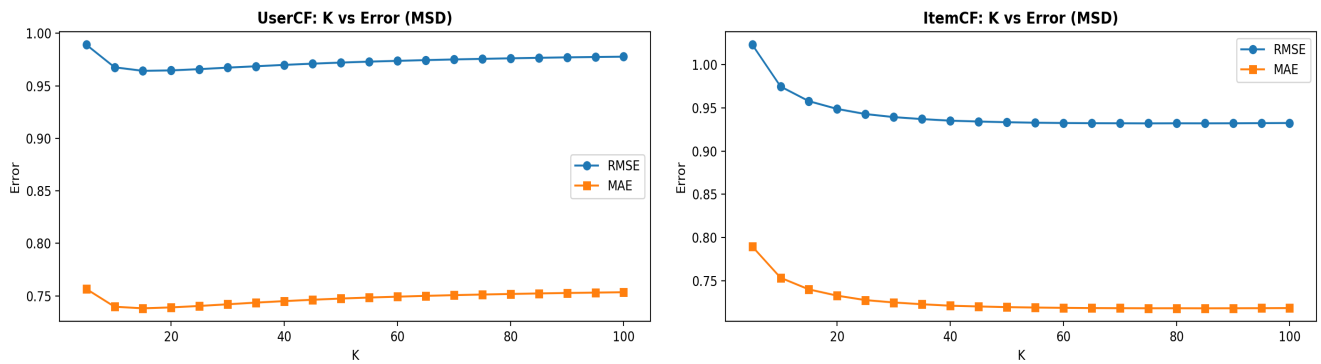


Figure 10: RMSE and MAE vs K — UserCF and ItemCF combined (Q2f/g)



Left: Figure 11: UserCF Neighbor Detail | Right: Figure 12: ItemCF Neighbor Detail

Answer: For **UserCF**: performance improves as K increases from 5, peaks around K=15, then degrades steadily as larger neighborhoods include dissimilar users that dilute the signal. For **ItemCF**: RMSE improves consistently as K grows from 5 to ~75, then plateaus. Item profiles are more homogeneous — a film's co-rated items share consistent taste patterns — so more neighbors continue to add reliable signal well beyond K=15. Both RMSE and MAE curves confirm this behavioral difference. **Per-method K tuning is therefore essential.**

Q2g (10 pts): Best K per Method

CF Method	Best K (by RMSE)	Best RMSE	Best MAE
UserCF (MSD)	15	0.9643	0.7381
ItemCF (MSD)	75	0.9322	0.7180

Answer: The best K is not the same for the two methods. UserCF peaks at K=15 (RMSE=0.9643), while ItemCF peaks at K=75 (RMSE=0.9322). Movie rating profiles (items) are more homogeneous — a film's audience shares consistent taste patterns — so a larger neighborhood continues to provide reliable signal. User profiles are more heterogeneous, so a tighter neighborhood avoids including dissimilar users. This confirms that per-method hyperparameter tuning cannot be shared between UserCF and ItemCF.

Overall Summary

Task 1 — K-Means (9,999 samples × 784 features, K=10):

Euclidean produces the lowest SSE and the most stable convergence (true fixed point at 66 iterations). Cosine achieves the highest accuracy (64.45%) by capturing digit shape independently of brightness. Jaccard is competitive on both metrics. Stopping condition has no measurable effect on Euclidean but produces small differences for Cosine and Jaccard. Core takeaway: match the distance metric to the evaluation criterion — they rank differently for SSE vs accuracy.

Task 2 — Recommender Systems (MovieLens ratings_small.csv):

ItemCF with Cosine similarity at K≈75 is the top configuration on this dataset. Both CF methods dramatically outperform PMF with default settings. The best similarity metric differs by CF type (MSD for UserCF, Cosine for ItemCF), and the optimal K also differs (UserCF=15, ItemCF=75), confirming that per-method tuning is essential for recommender systems.

Method	Best Config	Best RMSE	Best MAE
UserCF	MSD, K=15	0.9643	0.7381
ItemCF	Cosine, K=75	0.9322	0.7180
PMF	20 factors, SGD	1.1292	0.8313

```
Code Repository: https://github.com/nikhilsrikant/Data-Mining-HW3
task1_kmeans.py | task2_recommender.py
```